矩陣理論及應用　期末專題

盲式數位浮水印的 JPEG 抗壓縮分析與 SVD 區塊軟判決

內容審閱稿　G3 結果待填

作者　林易辰
學號　B143012007
學校／系級　〔待填〕
授課教師　〔待填〕
繳交日期　〔待填〕

摘要

同一張含浮水印影像經 JPEG 壓縮後，外觀看起來仍可能相似，擷取的浮水印位元卻可能出錯。本專題以 blind_watermark 的 DWT–DCT–SVD 盲擷取流程為對象，說明原版如何將位元寫入區塊的前兩個奇異值，以及如何以餘數門檻讀回。本文提出一項擷取端修改，以連續分數取代區塊內的兩個二元判決，保留原版嵌入、融合權重及最後分群。在相同含水印影像和 JPEG 解碼像素上，以逐位錯誤率比較兩種擷取方式。方法效果由 G3 配對實測評估，結果另章呈現。

關鍵字　盲式數位浮水印、JPEG、奇異值分解、軟判決、位元錯誤率

1 緒論

1.1 研究動機

將照片寄出或上傳後，平台可能重新儲存為 JPEG。觀看者仍可辨認原來的照片，但每個像素的值不一定與上傳前相同。若照片裡另有一段用來識別來源的浮水印，判斷浮水印是否保留下來，需要重新讀取並比對位元。單憑視覺相似度，無法知道其中有幾位被讀錯。

本研究使用開放原始碼 blind_watermark 的固定版本 [1, 2]。原版以矩陣分解取得區塊的奇異值，藉由改變奇異值的餘數寫入位元。擷取時只看餘數落在哪一側，便把該奇異值的判斷化成 0 或 1。假如兩個餘數都落在同一側，其中一個緊鄰分界，另一個靠近嵌入時的理想位置，原版仍給出相同的判斷值。這正是本專題選擇修改的步驟。

1.2 研究問題與範圍

在同一含水印 PNG、同一 JPEG 攻擊及同一組解碼像素上，保留區塊奇異值餘數的連續資訊，能否降低原版盲擷取的位元錯誤率？本研究只更換區塊內的奇異值判分。嵌入端及 JPEG 編碼設定固定，實驗時兩個擷取器讀取同一個檔案。如此可把觀察到的差異對應到解碼流程中的這一項修改。

先前 W0 已建立原版重現流程，W0-D 增加不同壓力條件，G2 比較嵌入步長 [4, 5, 6]。這些資料分別提供基準、失效情況及參數比較的紀錄。本稿以它們說明實驗設計的來源，軟判決的效果則由 G3 另行評估。

2 數位浮水印與 JPEG 基礎

2.1 嵌入與盲擷取

數位浮水印將短訊息寫入影像。本文把訊息表示為一串 0 和 1。設原圖為 I，訊息為 $b = (b_1, \ldots, b_L)$，嵌入後的影像為 I_w。JPEG 編解碼後得到 $\widehat{I}_{w,Q}$，其中 Q 是編碼器的品質設定。擷取器由受測影像、訊息長度、固定參數及密鑰取得估計位元 $\widehat{b}$。原圖及正確位元留在評估端，所以擷取屬於盲擷取。

$$(I, b) \longrightarrow I_w \xrightarrow{\text{JPEG 編解碼}} \widehat{I}_{w,Q} \xrightarrow{\text{盲擷取}} \widehat{b}.$$

無攻擊對照直接由存檔的 I_w 擷取。它用來檢查寫入、儲存、重讀及擷取的整條流程。若這一步已失敗，需先檢查基準，才能解讀 JPEG 攻擊下的差異。

2.2 JPEG 為何可能改變判決

本研究的 JPEG 是有損影像編碼。編碼器將影像轉為可壓縮表示，量化會捨去部分精度，解碼後的像素因此可能改變 [3]。JPEG 品質設定可用來建立不同攻擊條件，實驗仍須記錄實際的編碼器、色度取樣及其他設定。本文把 JPEG 視為黑盒，只控制輸入和編解碼條件，並讀回實際解碼影像。

浮水印程式本身也使用 DCT，但用途不同。它先在小區塊內整理資料，再從變換後的矩陣取得奇異值以讀寫浮水印。JPEG 改變影像像素後，擷取器重算的區塊矩陣及其奇異值可能改變。若餘數跨過判決邊界，部分區塊的票可能變成另一個位元。

2.3 位元與影像品質怎麼量

位元錯誤率（bit error rate，BER）是逐一比較位置所得的比例。以 $\mathbf{1}[\cdot]$ 表示條件成立時取 1，否則取 0，則

$$\mathrm{BER} = \frac{1}{L}\sum_{i=1}^{L} \mathbf{1}[\widehat{b}_i \neq b_i].$$

例如讀回 32 位而其中 2 位錯誤，BER 為 $2/32 = 6.25\%$。計算前先確認長度和位元順序，報告時同時保留錯誤數與總位元數。

影像品質以峰值訊雜比（peak signal-to-noise ratio，PSNR）描述。對同尺寸的 8 位元 RGB 影像 X, Y，令 N 為三色通道的樣本總數，則

$$\mathrm{MSE}(X,Y) = \frac{1}{N}\sum_{j=1}^{N}(X_j - Y_j)^2, \qquad \mathrm{PSNR}(X,Y) = 10\log_{10}\frac{255^2}{\mathrm{MSE}(X,Y)}.$$

均方誤差為零時，PSNR 記為 $+\infty$。$\mathrm{PSNR}(I, I_w)$ 描述嵌入失真，$\mathrm{PSNR}(I, \widehat{I}_{w,Q})$ 描述嵌入與 JPEG 的合併失真。兩者用不同欄位記錄。G3 的原版與軟判決擷取器共用相同像素，所以在同一影像與攻擊條件下，兩個 PSNR 數值也分別相同。

3 奇異值分解基礎

3.1 一個區塊如何寫成矩陣成分

用 B 表示一個送進 SVD 的實數 4×4 矩陣。其奇異值分解可寫成

$$B = U\Sigma V^{\mathsf{T}} = \sum_{j=1}^{4} \sigma_j u_j v_j^{\mathsf{T}}, \qquad \sigma_1 \geq \sigma_2 \geq \sigma_3 \geq \sigma_4 \geq 0.$$

U 與 V 的直行向量（columns）各自正交歸一，Σ 的對角元素是奇異值。以 $B^{\mathsf{T}}B$ 來看，σ_j 是其特徵值的非負平方根。外積 $u_j v_j^{\mathsf{T}}$ 給出一個秩一成分的形狀，σ_j 控制該成分的大小。

以 $B = \mathrm{diag}(5, 2)$ 為小型例子。兩個方向分別對應左上角與右下角。把第一奇異值由 5 改為 6，便在第一個方向上增加 1，而第二方向保持原值。實際區塊的方向通常並非座標軸，這個改量會分布在矩陣的多個位置。

3.2 修改奇異值如何改變區塊

保留 U, V，將前兩項改為 σ_1', σ_2' 時，重建的矩陣 B' 與 B 相差

$$B' - B = (\sigma_1' - \sigma_1) u_1 v_1^{\mathsf{T}} + (\sigma_2' - \sigma_2) u_2 v_2^{\mathsf{T}}.$$

兩個外積的 Frobenius 內積為 $(u_1^{\mathsf{T}} u_2)(v_1^{\mathsf{T}} v_2) = 0$，各自平方範數為 1。因此

$$\|B' - B\|_F^2 = (\sigma_1' - \sigma_1)^2 + (\sigma_2' - \sigma_2)^2.$$

這是固定方向下的區塊矩陣等式。經過逆變換、像素量化、裁切及 JPEG 後，整張圖的失真和位元錯誤率仍應從實際影像計算。

4 原版 DWT–DCT–SVD 演算法

4.1 從彩色影像到嵌入區塊

原版 common 模式將影像轉成 YUV 三通道，對每個通道做 Haar 離散小波轉換（DWT），取低頻近似部分，再切成 4×4 區塊。各塊做離散餘弦轉換（DCT），把 16 個係數展開並按密鑰產生的順序重排，還原成 4×4 矩陣後做 SVD。位元會在通道和區塊間重複寫入，以便在擷取時合併多次觀察 [1]。

程式把前兩個奇異值寫成 s[0]、s[1]。本文用 $j = 1, 2$ 對應，原版步長為 $d_1 = 36$, $d_2 = 20$。令要寫入該塊的位元為 $b \in \{0, 1\}$，則

$$\sigma_j' = d_j \left(\left\lfloor \frac{\sigma_j}{d_j} \right\rfloor + \frac{1}{4} + \frac{b}{2} \right), \qquad j = 1, 2.$$

整數部分保留原本所在的步長區間，小數位置則由位元決定。寫入 0 時理想餘數比例為 $1/4$，寫入 1 時為 $3/4$。例如 $\sigma_1 = 50$、$d_1 = 36$，寫入 0 得到 45，寫入 1 得到 63，餘數分別為 9 和 27。改值後重建區塊，依序還原係數順序、執行逆 DCT 和逆 DWT，最後還原彩色影像。

4.2 原版擷取如何把區塊判斷合成一位

擷取端重做相同區塊處理及係數重排。以 s_j 表示受測影像重算的第 j 個奇異值，令

$$r_j = \frac{s_j \bmod d_j}{d_j} \in [0, 1), \qquad h(r_j) = \mathbf{1}[r_j > 1/2].$$

因此恰等於 $1/2$ 時判為 0。原版把兩個硬判決以 $3:1$ 合成區塊分數 $(3h(r_1)+h(r_2))/4$。這個分數只可能取 $0,\ 1/4,\ 3/4,\ 1$。

對應同一訊息位置的區塊分數，跨重複位置與三通道取平均。公開的 mode="bit" 接著對整串平均分數做一維兩群 k-means，把高分群判為 1，再依密鑰還原位元順序 [1, 2]。所以 $1/2$ 是單個奇異值的餘數門檻。最後的位元判決由所有平均分數共同決定。

5 預定方法：SVD 區塊軟判決

5.1 把二元判斷改為連續分數

原版在 $r = 0.49$ 與 $r = 0.51$ 分別給 0 和 1，兩個輸入很近，輸出卻相差 1。擬議方法將 $h(r)$ 換成 G3 程式已固定的週期函數

$$g(r) = \frac{1 - \sin(2\pi r)}{2}.$$

它在嵌入 0 的理想位置 $r = 1/4$ 取 0，在嵌入 1 的理想位置 $r = 3/4$ 取 1；在 $r = 0$ 與 $1/2$ 取 $1/2$，且 r 從接近 1 繞回 0 時分數連續。靠近邊界的值在平均前保留為接近 $1/2$ 的分數。

餘數比例 r	原版硬判決 $h(r)$	軟分數 $g(r)$
0.25	0	0
0.49	0	約 0.469
0.50	0	0.500
0.51	1	約 0.531
0.75	1	1

表 1　由判分公式算出的示例值

表 1 顯示邊界兩側的輸出差距變小。這些數值只說明函數本身；它們不代表經 JPEG 實際量到的分數。軟分數也沒有經機率校準，0.531 不能解讀為該位元有 53.1% 的正確率。

5.2 改動位置與可驗證假說

每個區塊改用 $(3g(r_1) + g(r_2))/4$ 作為分數。DCT、係數重排、SVD、$d_1 = 36$、$d_2 = 20$、$3:1$ 權重、平均、原版 k-means 和位元順序還原均沿用原流程。程式僅替換軟判

決解碼器實例的 common 區塊擷取步驟。原版解碼器仍直接走上游路徑，兩者共享原版嵌入的影像。

預期效果是接近邊界的區塊較不會以極端分數左右平均，因而可能降低 BER。也可能出現反例，例如 JPEG 讓多數區塊的分數一起偏移，或分群整體的高低方向與真實位元相反。軟判決保留原有的分群及群方向規則，無法靠公式保證這些情況會改善。兩個擷取器都不使用正確位元修正群方向。

5.3 預定的 G3 配對檢驗

沿用 W0 的三張固定 RGB 影像、固定 32 位元訊息和密鑰。每張 W0 封存含水印 PNG 作無攻擊對照。JPEG 端使用 W0-D 由各圖含水印 PNG 獨立編碼並封存的 Q10、Q20、Q30 影像，色度取樣固定 4:4:4。依 W0-D 與 G2 的條件，Q10 為主要比較，Q20 為高壓診斷，Q30 為對照。對每張圖和每個攻擊條件，原版與軟判決擷取器讀入同一份解碼像素。報告須保留逐位輸出、錯誤數與分母，並逐圖比較

$$\Delta\mathrm{BER} = \mathrm{BER}_{\mathrm{soft}} - \mathrm{BER}_{\mathrm{hard}}.$$

負值代表該配對條件下軟判決錯得較少。若無攻擊條件已發生錯誤，應保留並說明。每個條件僅三張圖、同一訊息合計 96 位元，結果限於指定的資料和編碼設定。

6 實驗結果與討論

本章保留待填。屆時引用固定提交的逐位 CSV、manifest 與重跑證據，分別報告無攻擊、主要比較、高壓診斷及對照，並列出未改善與退步的條件。

參考資料

[1] Guofei，blind_watermark，固定提交的 bwm_core.py。

[2] Guofei，blind_watermark，同一提交的公開擷取介面。

[3] ITU-T，Recommendation T.81，JPEG 編碼標準。

[4] 專題倉庫，W0 基準 manifest。

[5] 專題倉庫，W0-D JPEG 壓力診斷。

[6] 專題倉庫，G2 第二奇異值步長比較。